

Daten laden via JDBC/ODBC und REST-APIs in Notebooks

Section 2 · Data Ingestion and Loading · Quelle: Databricks-Dokumentation (JDBC/ODBC)

Nicht jede Datenquelle lässt sich über einen Lakeflow-Connect-Connector oder Auto Loader anbinden. Für klassische relationale Datenbanken (SQL Server, PostgreSQL, MySQL, Oracle, ...) sowie für Systeme, die nur eine REST-API anbieten, ist es üblich, direkt aus einem Notebook heraus per **JDBC/ODBC** bzw. per HTTP-Client Daten zu laden und anschließend in Cloud-Speicher oder direkt in eine Unity-Catalog-verwaltete Tabelle zu schreiben.

1. Datenbanken per JDBC anbinden

Spark bringt einen generischen JDBC-Datenquellen-Connector mit. Damit lässt sich eine komplette Tabelle oder das Ergebnis einer beliebigen SQL-Abfrage direkt als DataFrame laden:

```
jdbc_url = "jdbc:postgresql://my-db-host:5432/salesdb"

df = (spark.read
      .format("jdbc")
      .option("url", jdbc_url)
      .option("dbtable", "public.orders")
      .option("user", dbutils.secrets.get(scope="db", key="user"))
      .option("password", dbutils.secrets.get(scope="db", key="password"))
      .option("driver", "org.postgresql.Driver")
      .load())

df.write.mode("overwrite").saveAsTable("main.bronze.orders_raw")
```

Wichtig für produktive Ingestion großer Tabellen: Ohne weitere Optionen liest Spark die gesamte Tabelle über eine einzige Verbindung/Partition – das skaliert schlecht. Mit `partitionColumn`, `lowerBound`, `upperBound` und `numPartitions` lässt sich der Lesevorgang auf mehrere parallele JDBC-Verbindungen aufteilen:

```
df = (spark.read
      .format("jdbc")
      .option("url", jdbc_url)
      .option("dbtable", "public.orders")
      .option("partitionColumn", "order_id")
      .option("lowerBound", "1")
      .option("upperBound", "1000000")
      .option("numPartitions", "8")
      .load())
```

Credentials sollten dabei – wie im Beispiel – nie im Klartext im Notebook stehen, sondern über **Databricks Secrets** (`dbutils.secrets.get(...)`) referenziert werden.

2. ODBC: dieselbe Aufgabe aus der anderen Richtung

Während JDBC in diesem Szenario verwendet wird, um *aus* Databricks heraus eine externe Datenbank anzuzapfen, wird der von Databricks bereitgestellte **ODBC-Treiber** meist in die andere Richtung genutzt: Externe BI-Tools (Power BI, Tableau) verbinden sich per ODBC/JDBC *zu* Databricks, um auf Unity-Catalog-Tabellen zuzugreifen. Für die Prüfung reicht das Verständnis, dass beide Treiber dem gleichen Zweck dienen – standardisierter Datenbankzugriff – nur aus jeweils entgegengesetzter Blickrichtung.

3. REST-APIs aus dem Notebook heraus abfragen

Für Systeme ohne JDBC-Schnittstelle (z. B. SaaS-Anwendungen mit REST-API) wird die Anfrage typischerweise mit Pythons Standard-HTTP-Bibliotheken direkt im Notebook ausgeführt und das JSON-Ergebnis anschließend in ein Spark-DataFrame überführt:

```
import requests

response = requests.get(
    "https://api.example.com/v1/customers",
    headers={"Authorization": f"Bearer {dbutils.secrets.get('api', 'token')}}"
)
response.raise_for_status()
records = response.json()["data"]

# In ein Spark-DataFrame überführen und als Delta-Tabelle schreiben
df = spark.createDataFrame(records)
df.write.mode("append").saveAsTable("main.bronze.customers_api_raw")
```

Solche REST-Ingestion-Notebooks werden in der Praxis so gut wie nie manuell gestartet, sondern als **Notebook-Task innerhalb eines Lakeflow Jobs** geplant – z. B. stündlich per Cron-Schedule, damit neue Datensätze regelmäßig automatisiert nachgeladen werden (siehe Section 4 zu Lakeflow Jobs).

4. Einordnung gegenüber anderen Ingestion-Methoden

Methode	Typischer Anwendungsfall
Lakeflow Connect (Managed Connector)	Verbreitete Enterprise-/SaaS-Quellen mit vorgefertigtem, wartungsarmem Connector
Auto Loader	Dateibasierte Quellen in Cloud-Speicher, kontinuierlich wachsend
JDBC/ODBC im Notebook	Klassische relationale Datenbanken ohne Managed Connector, einmalige oder geplante Batch-Abzüge
REST-Client im Notebook	SaaS-/Web-APIs ohne Datenbankschnittstelle und ohne verfügbaren Managed Connector

Ergänzt aus der offiziellen Databricks-Dokumentation: Databricks stellt sowohl einen offiziellen JDBC- als auch einen ODBC-Treiber bereit, die dem Industriestandard folgen und u. a. auch von BI-Tools zum Zugriff *auf* Unity-Catalog-Tabellen genutzt werden. Für den ODBC-Treiber ab Version 2.6.17 wird zusätzlich **Cloud Fetch** unterstützt: Große Abfrageergebnisse werden dabei über den ans Databricks-Deployment angebundenen Cloud-Speicher statt über eine einzelne Verbindung übertragen, was den Datendurchsatz bei großen Ergebnismengen deutlich erhöht.

Quelle: [Databricks ODBC and JDBC Drivers – Databricks-Dokumentation](#)